JVM Runtime Health Guide

Metrics, Garbage Collection, Thread Pools, Blocked Threads & Connection Pools

This guide is a practical field reference for diagnosing and monitoring JVM-based applications in production. It covers five interlocking areas: core JVM metrics, garbage collection behavior, thread pool tuning, blocked-thread diagnosis, and connection pool management. Each section includes what to watch, healthy ranges, common failure patterns, and how to inspect them using standard tooling (JMX, **jstat**, **jstack**, **jcmd**, and Micrometer/Prometheus-style metrics).

---

1. Core JVM Metrics

These are the baseline signals every JVM service should expose. Most are available via JMX MBeans (**java.lang:type=Memory**, **Threading**, **OperatingSystem**) and are automatically surfaced by Micrometer's **JvmMemoryMetrics**, **JvmGcMetrics**, and **JvmThreadMetrics** binders in Spring Boot / Dropwizard style stacks.

1.1 Memory Metrics

Metric	What it means	Healthy signal
Heap used / max	Live objects + garbage awaiting collection vs. -Xmx ceiling	Sawtooth pattern; used heap returns to a stable baseline after each major GC
Heap committed	Memory the OS has actually allocated to the JVM for heap	Should not constantly climb toward -Xmx without corresponding GC relief
Non-heap (Metaspace)	Class metadata, method bytecode, constant pools	Stabilizes after warm-up; continuous growth suggests classloader leaks
Old Gen occupancy	Long-lived / promoted objects	Stays well under threshold that triggers full GC
GC pause time	Stop-the-world duration per collection	p99 pause under your latency SLO (often <200ms)
Allocation rate	Bytes/sec allocated in young gen	Stable under steady load; spikes indicate churn or leaks

1.2 Process & Runtime Metrics

- **CPU usage (process vs. system):** sustained near 100% often correlates with GC thrashing or busy-spin locks, not just legitimate load.
- **Class loading count:** a continuously rising loaded-class count without matching unloads points to classloader leaks (common with hot-redeploy app servers).
- **JIT compilation time:** spikes during warm-up are normal; recurring spikes in steady state suggest deoptimization thrashing.
- **Open file descriptors / native memory (NMT):** critical for diagnosing native leaks outside the heap (direct ByteBuffers, native libraries).

1.3 Inspecting Live

```
# Snapshot of heap regions and GC counts
jstat -gc <pid> 1000 5

# Quick memory summary
jcmd <pid> GC.heap_info

# Native memory tracking (requires -XX:NativeMemoryTracking=summary at startup)
jcmd <pid> VM.native_memory summary
```

Watch for: a heap graph that never returns to baseline after major GCs — this is the classic signature of a memory leak, not just memory pressure.

2. Garbage Collection

GC behavior is usually the single biggest driver of tail latency in JVM services. Understanding which collector is active and its generational behavior is essential before tuning anything.

2.1 Common Collectors

Collector	Profile	Best for
Serial	Single-threaded, stop-the-world	Small heaps, CLI tools, constrained containers
Parallel (Throughput)	Multi-threaded STW collections	Batch jobs prioritizing throughput over pause time
G1 (Garbage First)	Region-based, mostly concurrent, predictable pause targets	Default since JDK 9; most general-purpose services
ZGC	Concurrent, sub-millisecond pauses, scales to very large heaps	Latency-critical services, large heaps (10s–100s of GB)
Shenandoah	Concurrent compaction, low pause times	Similar use case to ZGC; OpenJDK/RedHat-maintained

2.2 Key GC Signals

- **Minor (young) GC frequency:** frequent young GCs are normal and cheap; a sudden increase in frequency signals a rise in allocation rate.
- **Major/Full GC occurrence:** full GCs are expensive and should be rare in a healthy G1/ZGC setup. Frequent full GCs indicate an undersized old generation or a leak.
- **Promotion rate:** how fast objects survive young GC into old gen. High promotion rate accelerates old-gen fill-up and triggers more major GCs.
- **GC throughput:** percentage of time NOT spent in GC. Below ~95% throughput usually warrants investigation.
- **Pause time distribution:** look at p50/p95/p99, not just averages — a few long pauses can dominate tail latency even if the mean looks fine.

2.3 Diagnosing GC Issues

```
# Enable structured GC logging (JDK 9+)
-Xlog:gc*,gc+heap=debug:file=gc.log:time,uptime,level,tags:filecount=10,filesize=50m

# Live GC cause/duration stream
jstat -gcutil <pid> 1000

# Force and inspect a full GC (use cautiously, not in prod hot path)
jcmd <pid> GC.run
```

Common anti-pattern: tuning heap size or GC flags before checking allocation hotspots. Reducing allocation pressure in code (object pooling, avoiding needless boxing, streaming instead of buffering) often outperforms GC flag tuning.

3. Thread Pools

Thread pools (java.util.concurrent.ExecutorService, servlet container pools, DB connection dispatchers) are the primary concurrency control point in most JVM services. Misconfigured pools are one of the most common causes of cascading outages.

3.1 Metrics to Track per Pool

Metric	Meaning	Warning sign
Active threads	Threads currently executing a task	Pinned at max size for sustained periods
Pool size vs. max	Current vs. configured ceiling	Frequently at max under normal (non-peak) load
Queue size	Tasks waiting for a free thread	Continuously growing = pool undersized or tasks too slow
Rejected task count	Tasks refused when pool + queue are full	Any non-zero rate in production is a real problem
Task latency (queue + execution)	Total time from submission to completion	Rising queue-wait component before execution time rises
Completed task count / rate	Throughput of the pool	Sudden drop while active threads stay high

3.2 Sizing Guidance

- **CPU-bound work:** size ≈ number of CPU cores (+1), since more threads than cores just adds context-switch overhead.
- **I/O-bound work:** size can exceed core count substantially; a common starting formula is *threads = cores × (1 + wait time / service time)*.
- **Bounded queues:** prefer a bounded queue with a sane rejection policy (e.g. **CallerRunsPolicy** for backpressure) over an unbounded queue, which just delays an OOM instead of failing fast.
- **Isolate pools by workload:** don't share one pool between latency-sensitive request handling and slow background jobs — one starves the other.

3.3 Inspecting Live

```
# Thread counts and states via JMX (ThreadMXBean) or jcmd
jcmd <pid> Thread.print

# Programmatic pool inspection (ThreadPoolExecutor)
executor.getActiveCount(); // busy threads
executor.getPoolSize(); // current threads
executor.getQueue().size(); // pending tasks
executor.getCompletedTaskCount();
```

4. Blocked Threads & Deadlocks

A thread in **BLOCKED** state is waiting to acquire a monitor lock held by another thread. Persistent or growing blocked-thread counts are one of the clearest early-warning signs of contention that will eventually manifest as latency spikes or full outages.

4.1 Thread States

State	Meaning
RUNNABLE	Executing or ready to execute on CPU
BLOCKED	Waiting to enter a synchronized block/method held by another thread
WAITING	Waiting indefinitely for another thread (Object.wait(), Thread.join())
TIMED_WAITING	Waiting with a timeout (sleep, wait(timeout), lock with timeout)
TERMINATED	Finished execution

4.2 What to Monitor

- **Count of threads in BLOCKED state over time:** a baseline of near-zero is expected; sustained non-zero counts indicate lock contention.
- **Monitor deadlock detection:** ThreadMXBean's **findDeadlockedThreads()** should be polled periodically and alerted on immediately if non-empty.
- **Lock contention hotspots:** repeated stack traces blocked on the same monitor/lock object across multiple thread dumps indicate a specific synchronization bottleneck.
- **Thread dump cadence during incidents:** capture 3–5 thread dumps, 10 seconds apart, during a suspected hang — comparing them reveals whether threads are truly stuck or just slow.

4.3 Diagnosing

```
# Full thread dump with lock ownership info
jstack -l <pid> > threads.txt

# Or via jcmd (preferred on modern JDKs)
jcmd <pid> Thread.print -l

# grep for blocked threads and what they're waiting on
grep -A5 "BLOCKED" threads.txt
```

Reading a thread dump: look for a thread marked "waiting to lock <0x...>" and cross-reference that same lock address against the thread that shows "locked <0x...>" — that pair is your contention chain. A cycle among such pairs is a deadlock.

5. Connection Pools

Connection pools (HikariCP, Apache DBCP, R2DBC pools, HTTP client pools) manage a finite, expensive resource — network connections to databases or downstream services. Pool exhaustion is one of the most common root causes of end-to-end request timeouts.

5.1 Metrics to Track

Metric	Meaning	Warning sign
Active connections	Connections currently checked out and in use	Pinned near max pool size continuously
Idle connections	Connections available for immediate reuse	Near zero while active is near max
Pending / waiting requests	Threads waiting for a connection to free up	Any sustained non-zero value
Connection acquisition time	Time to obtain a connection from the pool	Rising acquisition time before query time even changes
Connection lifetime / age	How long a connection has been open	Very old connections may be stale or leaking on servers that recycle sockets
Timeout / creation failure count	Failed attempts to acquire or create a connection	Any non-zero rate correlates directly with user-facing errors

5.2 Common Failure Patterns

- **Pool exhaustion cascades:** a slow downstream (e.g. a locked DB row) holds connections longer, which starves the pool, which queues more requests, which increases latency further — a classic feedback loop.
- **Connection leaks:** code paths that acquire a connection but fail to release it on an exception path slowly drain the pool until it's exhausted, often hours after deployment.
- **Pool sized larger than the backend can support:** an oversized pool can overwhelm the database's own max-connections limit, causing failures at the database rather than the pool.
- **Mismatched pool and thread-pool sizing:** a connection pool smaller than the request thread pool means threads will regularly queue for connections even under moderate load.

5.3 Inspecting Live (HikariCP example)

```
# HikariCP exposes a JMX MBean per pool: com.zaxxer.hikari:type=Pool (<poolName>)
# Key attributes:
ActiveConnections
IdleConnections
ThreadsAwaitingConnection
TotalConnections

# Micrometer auto-binds these as:
hikaricp_connections_active
hikaricp_connections_idle
hikaricp_connections_pending
hikaricp_connections_timeout_total
```

Sizing rule of thumb: HikariCP's own guidance is that pool size should usually be small — *connections* ≈ *cores* × *2* + *effective_spindle_count* — because more connections than the database can execute in parallel just adds queuing at the database instead of eliminating it.

Putting it together: these five areas are rarely independent in practice. A slow downstream call exhausts a connection pool → request threads block waiting for connections → the thread pool queue grows → allocation rate rises as retries pile up → GC frequency increases → pause times grow → latency compounds further. When triaging an incident, checking all five in this order (connection pools → thread pools/blocked threads → GC → memory) usually surfaces the root cause fastest.